

Génération de SPARQL sur Wikidata par few-shot prompting et sélection guidée par exécution : participation au défi WikiKGQA @ ISWC 2026*

Kenmegne Fokam Emeric Cyrille^{1,*}, Tela Tela Gilbert Ebenezer¹

¹ Université de Yaoundé I, Faculté des Sciences, BP 812 Yaoundé, Cameroun

Résumé

Nous décrivons notre participation aux deux pistes *with-mentions* (anglais et espagnol) du défi WikiKGQA @ ISWC 2026, dont la tâche consiste à traduire une question en langue naturelle en une requête SPARQL exécutable sur un sous-ensemble de Wikidata. Notre système enchaîne quatre étapes : un retrieval TF-IDF qui sélectionne trois exemples similaires dans le train, un *prompt few-shot* adressé à un LLM ouvert, une génération de cinq candidats à températures mixtes (un déterministe, quatre à 0,7) et une sélection du premier candidat renvoyant un résultat non vide sur l'*endpoint* du défi. Sur le développement (100 questions par langue), nous obtenons F1 macro = 0,634 en anglais et 0,607 en espagnol ; sur le test officiel Codabench (75 questions par langue), les scores atteignent 0,65 et 0,63. Une analyse d'erreurs isole deux catégories résistantes : les questions typées en « *which / qué* », qui exigent la chaîne P31/P279* incomplète sur l'*endpoint*, et les requêtes COUNT, pour lesquelles le LLM omet fréquemment l'agrégat. L'ensemble n'utilise que des modèles à pondérations ouvertes (*gpt-oss-120b*, *llama-3.3-70b*) accessibles gratuitement via Groq Cloud.

Keywords

Knowledge Graph Question Answering, Text-to-SPARQL, Wikidata, Large Language Models, In-context Learning, QAWiki

1. Introduction

Le *Knowledge Graph Question Answering* (KGQA) vise à interroger un graphe RDF en langue naturelle. Sur un graphe collaboratif comme Wikidata [1], la difficulté est double : désambiguïser les mentions vers les bons Q-identifiants, puis traduire la structure logique de la question en un motif de graphe cohérent — choix des propriétés, gestion des agrégats, filtres et jointures — le tout dans un contexte multilingue. Le défi WikiKGQA @ ISWC 2026 fournit 497 paires bilingues (anglais / espagnol) question-SPARQL adossées au corpus QAWiki, un *endpoint* QLever [2] servant une vue simplifiée de Wikidata figée pour toute la durée du défi, et une évaluation *online* sur Codabench utilisant la F1 macro QALD [3]. Nous participons aux deux pistes *with-mentions*, dans lesquelles les Q-identifiants des entités mentionnées sont fournis avec la question, ce qui isole la difficulté de la génération de la requête. Nos contributions sont : (i) un pipeline en quatre étapes — retrieval TF-IDF, *few-shot prompting*, génération multi-candidate, sélection guidée par exécution — entièrement construit sur des LLM ouverts ; (ii) une évaluation dev et test des deux pistes, avec F1 macro de 0,634 / 0,607 sur le dev et 0,65 / 0,63 sur le test ; (iii) une analyse d'erreurs structurée qui isole les deux catégories de questions les plus résistantes, à savoir les questions typées et les requêtes COUNT.

. WikiKGQA Challenge @ ISWC 2026 : International Semantic Web Conference, October 2026, Nara, Japan

*. Papier de participation au défi WikiKGQA @ ISWC 2026, pistes *en-with-mentions* et *es-with-mentions*.

*. Auteur correspondant.

. ✉ emeric.kenmegne@facsciences-uy1.cm (K. F. E. Cyrille); gilbert.tela@facsciences-uy1.cm (T. T. G. Ebenezer)

.  0009-0001-4232-2361 (K. F. E. Cyrille)

.  © 2026 Copyright for this paper by its authors. Use permitted under Creative Commons License Attribution 4.0 International (CC BY 4.0).

2. Travaux connexes

La série QALD [3], migrée sur Wikidata avec QALD-10, a fixé le protocole d'évaluation encore utilisé aujourd'hui (F1 macro sur l'union des réponses). Les approches classiques par *parsers* sémantiques supervisés ou par *templates* obtiennent de bons résultats sur des distributions vues à l'entraînement mais généralisent mal aux reformulations nouvelles ; la génération *text-to-SPARQL* par un modèle seq2seq entraîné produit une syntaxe correcte mais souffre d'IRI hallucinées — le modèle génère des propriétés plausibles qui n'existent pas dans le graphe cible. Depuis GPT-3 [4], plusieurs travaux ont montré qu'un LLM instruit produit des requêtes SPARQL compétitives à partir d'une poignée d'exemples démontrés en contexte [5] : KAPING [6] injecte le voisinage du graphe dans le prompt, ChatKBQA [7] décompose le processus en génération conditionnelle puis récupération. Nous héritons de cette lignée le principe du *few-shot prompting* guidé par retrieval, mais choisissons délibérément un retrieveur non appris (TF-IDF [8]) pour rester reproductible à partir de ressources publiques. En parallèle, Wang *et al.* [9] ont popularisé l'idée qu'échantillonner plusieurs sorties et voter améliore la robustesse d'un LLM ; Self-Refine [10] étend cette famille en demandant au modèle de critiquer puis réécrire sa sortie. Notre sélection guidée par exécution s'inscrit dans cet esprit tout en évacuant l'étape critique : c'est l'*endpoint* SPARQL lui-même qui joue le rôle d'oracle en séparant les requêtes vides des requêtes exploitables.

3. Tâche et données

Le défi demande, pour chaque question en langue naturelle, la production simultanée d'une requête SPARQL exécutable et de l'ensemble des réponses attendues : liste de Q-identifiants pour les **SELECT**, valeur booléenne pour les **ASK**, entier pour les **COUNT**. Les organisateurs proposent quatre pistes : deux avec mentions d'entités fournies (EN et ES), deux sans. Notre soumission couvre les deux pistes *with-mentions*. Le corpus QAWiki est distribué au format JSON dérivé du schéma QALD. Chaque item réunit un identifiant, la question dans les deux langues, la liste des mentions d'entités (label suivi du Q-identifiant), la requête SPARQL de référence et la liste des réponses attendues (Fig. 1). La version 2026.2 utilisée dans cet article contient 497 questions, réparties selon les splits officiels en 317 train, 100 dev et 80 test (dont 75 effectivement scorées). Le graphe cible est servi par un *endpoint* QLever [2] spécifique au défi. Ce détail importe : contrairement à l'*endpoint* public de Wikidata, la vue servie est *simplifiée* et certaines hiérarchies, notamment la fermeture transitive P31/P279*, sont incomplètes ; une requête syntaxiquement parfaite peut donc renvoyer un résultat vide simplement parce que la propriété interrogée n'est pas matérialisée. Nous exploitons ce fait dans la sélection par exécution (Sect. 4). La métrique officielle est la F1 macro QALD [3] : pour chaque question, la F1 est la moyenne harmonique de la précision et du rappel calculés sur l'ensemble des Q-identifiants renvoyés, et le score global est la moyenne des F1 par question. Nous rapportons également, à des fins d'analyse, le nombre de questions parfaitement répondues et complètement ratées.

4. Système proposé

Notre pipeline, désigné P3 dans la suite, est linéaire et n'emploie ni agent ni re-ranker appris. Il prend en entrée une question et ses mentions et produit une requête SPARQL avec l'ensemble de ses réponses (Fig. 2). Le retrieval sélectionne, pour chaque question cible, les trois voisins les plus proches dans le train au sens du cosinus entre vecteurs TF-IDF [8] construits sur des *n*-grammes de mots avec $n \in \{1, 2\}$; une heuristique légère infère le type de requête cible (**ASK**, **COUNT**, **SELECT**) à partir des interrogatifs et privilégie, à similarité comparable, les voisins de même type. Le vectoriseur s'entraîne en moins d'une seconde sur les 317 questions du train. Le prompt est composé de trois blocs (Fig. 3) : une consigne système qui fixe le format attendu et interdit les déclarations **PREFIX** (elles seront ajoutées automatiquement à l'exécution) ; les

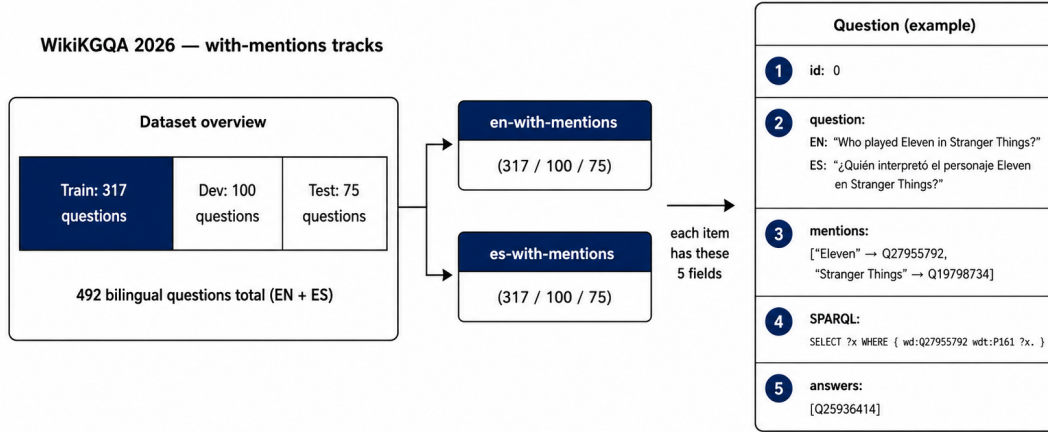


Figure 1: Format d'un item du corpus QAWiki : question bilingue, mentions d'entités avec Q-identifiants, requête SPARQL de référence, réponses attendues.

trois exemples formatés en triplets (**question**, **mentions**, **SPARQL**), chacun clos par un *fenced block* `“sparql ... “`; la question cible et ses mentions sous le même format. Pour chaque question, nous interrogeons le LLM cinq fois : un premier appel à température nulle (pari le plus probable) puis quatre appels à température 0,7 (diversification). Chaque candidat est extrait de son bloc de code, préfixé automatiquement par les déclarations **PREFIX** usuelles de Wikidata, puis soumis à l'*endpoint* via une requête HTTP avec un délai maximum de 30 secondes ; les codes 429 et 5xx sont gérés par une couche de *retry* adaptatif respectant le **Retry-After** de Groq. La règle de sélection est simple : nous parcourons les candidats dans l'ordre (déterministe d'abord, puis stochastiques) et renvoyons les réponses du premier candidat qui produit au moins une ligne. Si tous les candidats renvoient un ensemble vide, nous conservons la sortie déterministe, ce qui couvre les rares cas où la réponse de référence est effectivement vide (**ASK** négatif, **SELECT** avec **MINUS**). Le pipeline ne différencie pas les deux langues au niveau du code : le retrieveur, le format du prompt et la logique de sélection sont identiques ; TF-IDF construit toutefois deux vocabulaires distincts, un par langue.

5. Protocole expérimental

Nous utilisons les splits officiels (317 train, 100 dev, 75 test) et évaluons avec la F1 macro QALD définie plus haut. Deux LLM à pondérations ouvertes servent nos expériences : `openai/gpt-oss-120b` [11] et `meta-llama/llama-3.3-70b-versatile`, tous deux accessibles via le *free tier* de Groq Cloud [12] avec un quota quotidien respectivement de 200 k et 100 k tokens. Les hyperparamètres sont fixés à $K = 3$ voisins retrouvés, $N = 5$ candidats générés, séquence de températures (0,0; 0,7; 0,7; 0,7; 0,7), un plafond de 2048 tokens par appel et un délai de 6 secondes entre appels pour respecter la limite par minute de Groq. Le baseline de référence est la variante *single-shot* de P3, dans laquelle seul le premier candidat déterministe est produit ; il permet d'isoler l'apport strict de la génération multi-candidate. Nous n'incluons pas de comparaison à un *parser* sémantique appris car aucun baseline entraîné n'est fourni par les organisateurs, et notre objectif est de caractériser un pipeline entièrement *prompt-based*. Toutes les expériences se sont déroulées sur un ordinateur portable standard sans GPU, la charge d'inférence étant reportée sur les serveurs de Groq. Un *dev pass* complet prend de l'ordre de 15 à

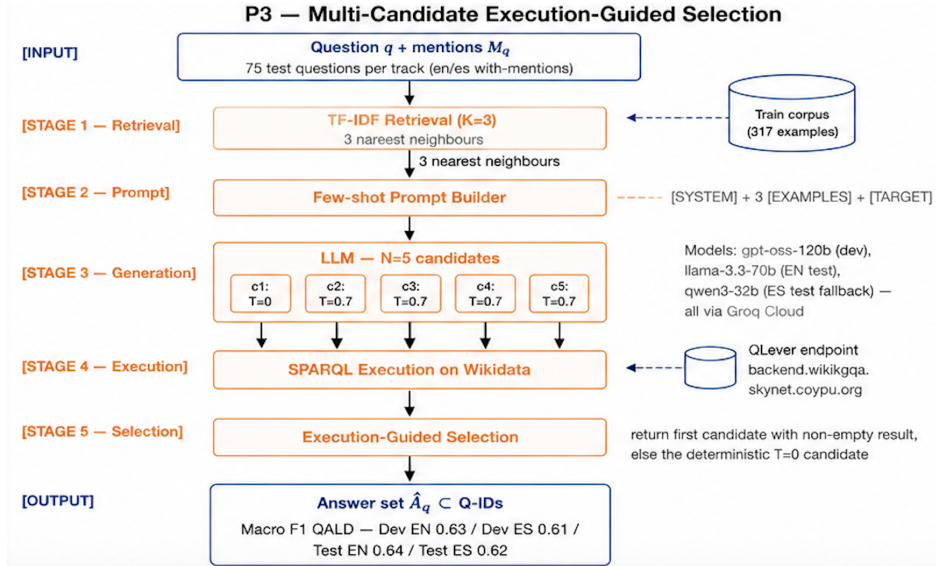


Figure 2: Pipeline P3. (1) Retrieval TF-IDF des trois voisins les plus proches ; (2) construction du *prompt few-shot* ; (3) génération de cinq candidats SPARQL par le LLM (un à température 0, quatre à 0,7) ; (4) exécution de chaque candidat sur l’*endpoint* du défi ; (5) sélection du premier candidat non vide.

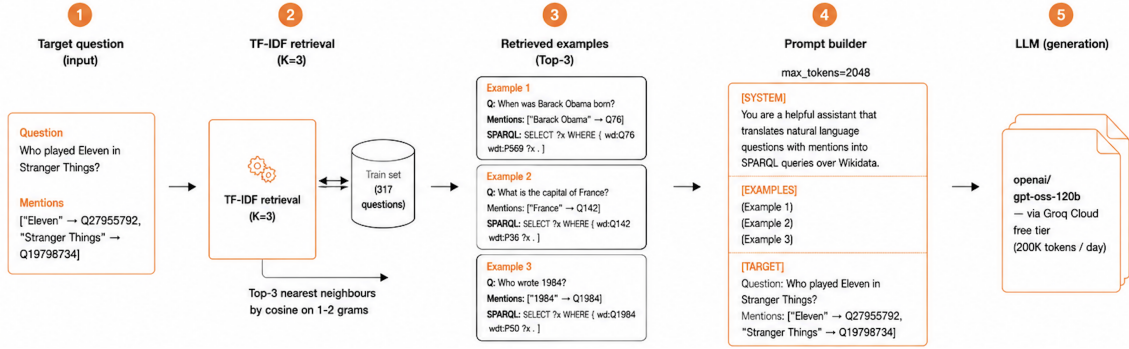


Figure 3: Construction du *prompt few-shot*. La question cible et ses mentions déclenchent un retrieval TF-IDF sur les 317 exemples du train ; les trois voisins retenus sont concaténés à la consigne système et à la cible avant envoi au LLM.

20 minutes, dont l’essentiel est absorbé par le *pacing* inter-appels. Un pré-test a vérifié que notre implémentation locale de la métrique reproduit exactement le score renvoyé par Codabench sur les soumissions dev, ce qui autorise à itérer localement avant chaque upload.

6. Résultats

La table 1 rassemble les scores sur le développement et sur le test officiel Codabench, et la Fig. 4 en propose une lecture visuelle. Le multi-candidate apporte un gain de +8,2 points de F1 en anglais et de +1,6 point en espagnol ; l’asymétrie s’explique par la borne inférieure : en espagnol, le candidat déterministe seul atteint déjà 0,59, ce qui laisse moins de marge de rattrapage. Les scores test (0,65 en anglais, 0,63 en espagnol) sont en parité étroite avec ceux du dev, ce qui indique l’absence de surapprentissage à la distribution de développement.

Pour quantifier la fragilité du retrieveur aux paraphrases, nous mesurons également le chevauchement Jaccard entre les voisinages top-3 renvoyés par TF-IDF et ceux de trois encodeurs de phrases modernes (all-MiniLM-L6-v2, bge-small-en-v1.5, bge-m3 [13]). Le Jaccard moyen oscille autour de 0,20 dans les trois cas, ce qui signifie qu’un remplacement par un retrieveur neuronal reconstruirait massivement le contexte *few-shot* ; c’est notre principale

Table 1

F1 macro sur le développement (100 questions) et le test officiel (75 questions) des deux pistes *with-mentions*. La variante *single-shot* sert de borne inférieure (seul le candidat déterministe est produit).

Piste	Méthode	F1 macro	Parfait	Nul
EN dev	Single-shot (N=1)	0,5524	52	40
EN dev	P3 (multi-candidate + exécution)	0,6342	60	31
ES dev	Single-shot (N=1)	0,5914	55	37
ES dev	P3 (multi-candidate + exécution)	0,6073	56	34
EN test	P3 (llama-3.3-70b)	0,65	—	—
ES test	P3 (gpt-oss-120b)	0,63	—	—

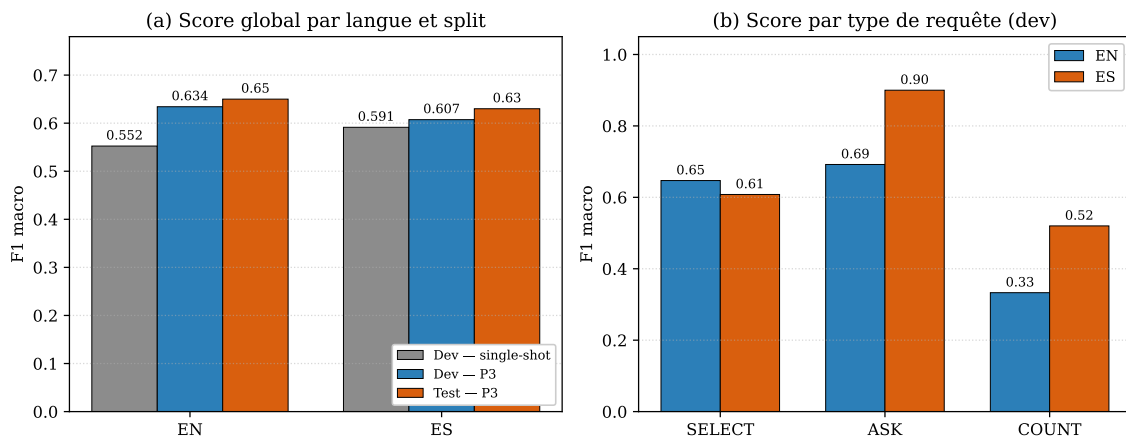


Figure 4: Résultats de P3. (a) F1 macro par langue et par split : le multi-candidate est plus rentable en anglais qu’en espagnol, et le test est en parité étroite avec le dev. (b) F1 par type de requête sur le développement : les ASK sont bien traités (spécialement en espagnol), les COUNT restent le point faible.

piste d’amélioration.

7. Discussion

Nous croisons les prédictions de P3 avec le type de requête généré et le mot interrogatif dominant (Table 2). Les COUNT sous-performent dans les deux langues ; le mode d’échec récurrent est l’oubli de l’agrégat, le LLM produisant `SELECT ?c WHERE { ... }` au lieu de `SELECT (COUNT(?c) AS ?n) WHERE { ... }`. À l’inverse, les ASK en espagnol atteignent 0,90 : les formulations oui/non en espagnol (« *¿Es ... ?* », « *¿Tiene ... ?* ») sont plus formellement contraintes qu’en anglais, ce qui guide bien le modèle. L’observation la plus nette est structurellement identique dans les deux langues : une seule catégorie d’interrogatifs concentre la moitié des nuls. En anglais, les « *which* » pèsent 32 % du dev et 15 nuls sur 31 ; en espagnol, les « *qué* » pèsent 39 % et 16 nuls sur 34. Ces deux catégories partagent la même structure logique : filtrer un ensemble d’entités par leur type, ce qui passe par la chaîne `?x wdt:P31/wdt:P279* wd:Qtype` ; or, cette fermeture transitive est incomplète sur l’*endpoint* du défi. Le LLM génère la chaîne plausible mais l’exécution renvoie zéro ligne. À l’inverse, les questions courtes reposant sur une propriété directe (« *when / cuándo* », « *who / quién* ») sont quasi-parfaites parce qu’elles reposent sur des propriétés bien matérialisées comme P569 (date de naissance) ou P161 (distribution). Deux gradients complémentaires confirment ce diagnostic : la F1 chute d’environ 0,30 point entre les questions à deux mentions et celles à trois ou plus, et de 0,16 point entre les questions courtes (≤ 6 mots) et moyennes (7–10 mots), les deux effets convergeant avec la catégorie « *which / qué* » plus longue et plus contrainte en moyenne. Enfin, la rentabilité du multi-candidate se répartit

en trois régimes : environ 75 % des questions sont résolues dès le premier tir ; les essais 2 à 4 rattrapent efficacement en anglais (F1 moyenne autour de 0,87) mais peu en espagnol (F1 moyenne autour de 0,30) ; lorsque les cinq candidats renvoient tous vide, le rattrapage est nul et ces 16 à 18 questions par langue correspondent au profil homogène des chaînes P31/P279*. Trois directions se distinguent pour la suite : un retrieveur neuronal (**bge-m3**) pour capter les paraphrases ; un profilage léger de l'*endpoint* — une requête `SELECT ?p (COUNT(?o) AS ?n)` par entité mentionnée — pour fournir au LLM la liste des propriétés effectivement disponibles ; un tour de raffinement conditionnel à la Self-Refine [10] sur les seuls cas où les cinq candidats renvoient vide.

Table 2

Performance par type de requête (bloc supérieur) et par mot interrogatif dominant (bloc inférieur) sur les 100 questions du dev EN et ES.

Catégorie	EN			ES		
	<i>n</i>	F1	Nul	<i>n</i>	F1	Nul
<i>Par type de requête</i>						
SELECT	81	0,647	23	74	0,608	24
ASK	13	0,692	4	10	0,900	1
COUNT	6	0,333	4	13	0,520	6
<i>Par mot interrogatif dominant</i>						
which / qué	32	0,425	15	39	0,470	16
what / cuál	23	0,701	6	9	0,667	3
who / quién	13	0,822	1	13	0,800	2
when / cuándo	9	1,000	0	9	0,889	1
how-many / cuántos	5	0,200	4	10	0,400	6
yes-no	13	0,692	4	11	0,818	2

8. Conclusion

Nous avons présenté un pipeline volontairement simple pour la conversion question-à-SPARQL sur Wikidata, appliqué aux deux pistes *with-mentions* du défi WikiKGQA @ ISWC 2026. En enchaînant un retrieval TF-IDF, un prompt *few-shot* de trois exemples, une génération de cinq candidats à températures mixtes et une sélection guidée par l'exécution sur l'*endpoint* du défi, le système atteint une F1 macro de 0,634 en anglais et 0,607 en espagnol sur le développement, et respectivement 0,65 et 0,63 sur le test officiel Codabench, en n'utilisant que des LLM à pondérations ouvertes accessibles gratuitement. L'analyse d'erreurs identifie deux catégories résistantes — les questions typées en « *which / qué* », contrariées par la fermeture transitive P31/P279* incomplète sur l'*endpoint*, et les requêtes `COUNT` pour lesquelles le LLM oublie l'agrégat — et ouvre trois pistes ciblées : un retrieveur neuronal pour capter les paraphrases, un profilage léger de l'*endpoint* pour éviter les propriétés non matérialisées, et un tour de raffinement conditionnel pour les questions où le multi-candidate a échoué à rattraper la réponse. Le code, les données et les archives Codabench sont publiés à l'adresse <https://github.com/emeric-cyrille/wikikgqa-iswc2026>.

Références

- [1] D. Vrandečić, M. Krötzsch, Wikidata : A free collaborative knowledgebase, Communications of the ACM 57 (2014).
- [2] H. Bast, B. Buchhold, Qlever : A query engine for efficient sparql+text search, in : CIKM, 2017.

- [3] R. Usbeck, A.-C. Ngonga Ngomo, B. Haarmann, A. Krithara, M. Röder, G. Napolitano, 7th open challenge on question answering over linked data (qald-7), in : SemWebEval @ ESWC, 2017.
- [4] T. Brown, B. Mann, N. Ryder, et al., Language models are few-shot learners, in : NeurIPS, 2020.
- [5] T. Li, X. Ma, A. Zhuang, Y. Gu, Y. Su, W. Chen, Few-shot in-context learning on knowledge base question answering, in : ACL, 2023.
- [6] J. Baek, A. F. Aji, A. Saffari, Kaping : Knowledge-augmented language model prompting for zero-shot knowledge graph question answering, in : NLRSE Workshop @ ACL, 2023.
- [7] H. Luo, H. E, Z. Tang, S. Peng, Y. Guo, W. Zhang, C. Ma, G. Dong, M. Song, W. Lin, Chatkbqa : A generate-then-retrieve framework for knowledge base question answering with fine-tuned large language models, in : ACL Findings, 2024.
- [8] G. Salton, C. Buckley, Term-weighting approaches in automatic text retrieval, Information Processing & Management 24 (1988).
- [9] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. H. Chi, S. Narang, A. Chowdhery, D. Zhou, Self-consistency improves chain of thought reasoning in language models, in : ICLR, 2023.
- [10] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegrefe, U. Alon, N. Dziri, S. Prabhume, Y. Yang, S. Gupta, B. P. Majumder, K. Hermann, S. Welleck, A. Yazdanbakhsh, P. Clark, Self-refine : Iterative refinement with self-feedback, in : NeurIPS, 2023.
- [11] OpenAI, gpt-oss-120b : Open-weight reasoning model, 2025. Open-weight release ; accessed via Groq Cloud.
- [12] Groq Inc., Groq cloud — inference platform for open-weight llms, 2024. <https://groq.com>.
- [13] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, Z. Liu, Bge m3-embedding : Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation, 2024. arXiv:2402.03216.